Unit testing CROSSFADE, Project 2: method and rationale

INFO 579 · info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh) · site: https://n-herling-mk1.github.io/INFO_579_Fa26_Proj1_CROSSFADE/

Why test at all

The brief asks for SQL, not tests. We added them on our own initiative so that every claim the four files make, in their comments and in their results, is checked by a machine rather than taken on trust. The premise: a query is right only if an independent computation of the same business question agrees with it, row for row.

The method

- **An independent oracle.** For every query and routine, plain Python with no SQL (*oracle.py*, *oracle_multiple.py*, *oracle_routines.py*) computes the expected answer from the Project 1 sample data (*data.csv*). MySQL must match it exactly: same columns, same order, same decimals.
- **Contract.** Each statement's comment is parsed: tag, use cases, business need, requirements claimed, output columns. A claim the SQL does not back fails, e.g. (d) declared without a LIMIT, or LIMIT used without (d).
- **Structure.** MySQL's own plan (EXPLAIN FORMAT=JSON) must read exactly the tables and columns the comment and its use cases declare; queries are read-only, never SELECT *, and name every join (INNER / LEFT OUTER JOIN).
- **Correctness and determinism.** The result equals the oracle's, and the ORDER BY must include a unique key, so the row order can never depend on chance.
- **Behaviour cases.** Edge cases the sample data lacks (ties, a play of exactly 30 s, a window of exactly 0.50, NULLs, a rejected upload, two uploads at once) are inserted inside a transaction; the query or routine is rerun and compared with the oracle; then everything is rolled back.
- **Coverage.** Each file as a whole: every non-associative table queried, every brief requirement met, every use case answered. database_implementation.sql is compared with the physical model through information_schema, every cell with data.csv, and every key and CHECK must refuse a bad write. Routines are compared with MySQL's catalogue, called, and pushed to their edges (each SIGNAL, the 1062 handler, a re-raised key clash).
- **Testing the tests.** A suite that never fails proves nothing, so *mutate_<file>.py* plants one deliberate bug at a time (a dropped tie-break, >= for >, LENGTH for CHAR_LENGTH, a missing filter, an undeclared column) and reruns the tests. A bug counts as caught when a passing test fails: **104/104 caught**.

Use cases, and how they are run

The 15 use cases (UC-01 to UC-15) come from the Project 1 requirements analysis (RA). Each is defined in *project2/data/use_cases.json* with the RA phrases that justify it (quoted word for word; the build fails if a phrase is not in the RA), the columns it needs, and the file that answers it. To run one: the query's comment names the use case; pytest runs the query on the *crossfade* database; the oracle computes that use case's answer from data.csv; the rows are compared; the use case's behaviour cases run in rolled-back transactions; and the results snapshot (*project2/data/<file>_results.json*) feeds the site. A query may read only the columns its use cases declare.

Run it, and the result

From the Deliverables page, a file's **file + tests** button downloads one folder with the file, its tests, their inputs, its rubric and the use case descriptions. Unzip it, then: `python -m venv .venv`; `.venv\Scripts\python -m pip install -r requirements.txt`; set `CROSSFADE_DB_PASSWORD`; `.venv\Scripts\python -m pytest --fresh` (database_implementation: no `--fresh`; it rebuilds the database itself). **429 tests** (228 / 71 / 80 / 50): **429 passed** in the repo, and **429 passed** in clean runs of the zips downloaded from the live site (2026-11-08, Windows, a new folder and fresh venv per zip).

Where to look (append to the site address)

`#p2/usecases`	use case definitions, their evidence from the RA, and the coverage map
`#p2/vet-implementation, #p2/vet-single, #p2/vet-multiple, #p2/vet-routines`	per file: the tests, coverage map, bugs planted and caught, and what the tests could miss
`#p2/map-implementation, #p2/map-single, #p2/map-multiple, #p2/map-routines`	each brief requirement mapped to the SQL that meets it
`#p2/rubric`	every condition of the brief with its evidence
`#p2/overview`	downloads and the record of tests run and passed
`repo`	project2/tests/ (tests, oracles), project2/docs/use_case_reference.pdf (every use case's input, expected output and test cases)